

Aprendizaje de Máquinas

Examen Primavera 2024

Índice de Contenidos

Índice de Figuras

1.	Enter Caption	16
2.	Enter Caption	16
3.		17

P1. (50 %) Preguntas Conceptuales

Responda un máximo de 8 de las siguientes preguntas.

- (a) Dado el modelo lineal $Y = \hat{X}\theta + \varepsilon$, ¿Qué interpretación geométrica tiene la **Regresión Lineal**? Hable sobre el **vector de errores** $\varepsilon = Y - \hat{X}\theta$ y su relación con $\text{span}(\hat{X})$, ¿a qué corresponde el **Estimador de Mínimos Cuadrados** en la relación anterior?

Respuesta:

Geométricamente, la Regresión Lineal busca proyectar el vector de observaciones $Y \in \mathbb{R}^n$ sobre el subespacio generado por las columnas de la matriz de diseño, es decir, sobre $\text{span}(\hat{X})$. El objetivo es encontrar el vector $\hat{X}\theta$ dentro de dicho subespacio que esté lo más cerca posible de Y en distancia euclidiana.

El vector de errores se define como

$$\varepsilon = Y - \hat{X}\theta.$$

Desde el punto de vista geométrico, este vector corresponde a la diferencia entre Y y su proyección ortogonal sobre $\text{span}(\hat{X})$. Por lo tanto, ε es perpendicular a todo vector en $\text{span}(\hat{X})$, lo cual se expresa mediante las ecuaciones normales:

$$\hat{X}^\top \varepsilon = 0.$$

El **Estimador de Mínimos Cuadrados Ordinarios** (OLS) corresponde exactamente a la elección de $\hat{\theta}$ tal que $\hat{X}\hat{\theta}$ sea la proyección ortogonal de Y sobre $\text{span}(\hat{X})$. Algebraicamente, esto se obtiene minimizando $\|\varepsilon\|^2$ y está dado por

$$\hat{\theta} = (\hat{X}^\top \hat{X})^{-1} \hat{X}^\top Y,$$

suponiendo que $\hat{X}$ tiene rango completo.

- (b) ¿Qué rol juega la elección de la **métrica de similitud o distancia** en un algoritmo de clustering? Compare la **distancia Euclidiana** y otra métrica a su elección. Explique como afecta la **escala de los datos** en el rendimiento de los algoritmos.

Respuesta:

La métrica de similitud o distancia determina cómo el algoritmo de *clustering* evalúa qué tan similares o diferentes son dos puntos. Por lo tanto, define la forma de los grupos y la manera en que se decide la asignación de cada observación a un clúster. En particular, distintas métricas pueden producir particiones completamente distintas sobre los mismos datos.

La **distancia Euclidiana** se define como

$$d_E(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}.$$

Esta métrica favorece clústers con forma aproximadamente esférica y es muy sensible a la escala de los datos, ya que variables con mayor rango influyen más en la distancia total.

Como comparación, consideremos la **distancia Manhattan**:

$$d_M(x, y) = \|x - y\|_1 = \sum_{i=1}^d |x_i - y_i|.$$

Esta métrica es menos sensible a valores extremos y tiende a producir regiones con geometría tipo “diamante”. Dependiendo del algoritmo (por ejemplo, **k-medians**), puede ser más robusta a *outliers*.

La **escala de los datos** influye fuertemente en el rendimiento de los algoritmos de clustering porque la mayoría de las métricas dependen directamente de las magnitudes de cada dimensión. Si una variable está en una escala mucho mayor que las demás, dominará el cálculo de distancias e inducirá clústers sesgados. Por ello, es habitual aplicar normalización o estandarización (por ejemplo, **z-score**) antes de ejecutar algoritmos basados en distancias, tales como **k-means** o **hierarchical clustering**.

- (c) Describa las **diferencias** entre un **modelo basado en densidad** como **DBSCAN** y un **modelo basado en distribuciones probabilísticas** como **GMM**.

Respuesta:

Un modelo basado en densidad como **DBSCAN** y un modelo basado en distribuciones probabilísticas como **GMM** difieren tanto en sus supuestos como en la forma en que construyen los clústers:

- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise):**
 - Identifica clústers como regiones de alta densidad separadas por regiones de baja densidad.
 - No requiere especificar el número de clústers; usa los parámetros ε (radio) y **minPts** (densidad mínima).
 - Puede detectar clústers de forma arbitraria (no necesariamente esféricos).
 - Clasifica puntos como *núcleo*, *frontera* o *ruido*, permitiendo identificar **outliers**.
 - No asume un modelo probabilístico subyacente.
- **GMM (Gaussian Mixture Models):**
 - Supone que los datos provienen de una mezcla de distribuciones Gaussianas.
 - Requiere especificar el número de componentes K .
 - Modela cada clúster mediante una distribución con su propia media y matriz de covarianza, permitiendo clústers elípticos.
 - Asigna a cada punto una **probabilidad** de pertenecer a cada clúster (clustering suave).
 - No detecta naturalmente puntos ruidosos; todos los puntos pertenecen a alguna componente.

En resumen, DBSCAN construye clústers en base a regiones densas sin asumir una forma específica ni un modelo probabilístico, mientras que GMM define los clústers a través de distribuciones Gaussianas y entrega asignaciones probabilísticas.

- (d) ¿Qué son las **medidas de impureza** en los árboles de decisión? Compare **entropía** e **impureza de Gini**.

Respuesta:

Las **medidas de impureza** en los árboles de decisión cuantifican qué tan mezcladas están las clases dentro de un nodo. El objetivo del algoritmo es elegir particiones que reduzcan la impureza, produciendo nodos cada vez más homogéneos. Dos de las medidas más comunes son la **entropía** y la **impureza de Gini**.

- **Entropía:**

$$H(p) = - \sum_{k=1}^K p_k \log p_k,$$

donde p_k es la proporción de la clase k en el nodo. Mide la incertidumbre del nodo: es 0 cuando el nodo es puro y máxima cuando las clases están uniformemente distribuidas. Se utiliza en el criterio **information gain**.

- **Impureza de Gini:**

$$G(p) = 1 - \sum_{k=1}^K p_k^2.$$

Representa la probabilidad de clasificar incorrectamente una muestra seleccionada aleatoriamente si se asigna la clase más frecuente del nodo. También es 0 en nodos puros y aumenta con la mezcla de clases.

Comparación: Ambas medidas tienen comportamientos muy similares: prefieren divisiones que generen nodos puros y penalizan la mezcla de clases. La entropía tiende a penalizar más las distribuciones muy mixtas (por su forma logarítmica), mientras que Gini suele ser más simple computacionalmente y ligeramente más conservadora. En la práctica, las diferencias en rendimiento entre ambas son mínimas.

- (e) ¿Qué establece el **Teorema de Aproximación Universal** sobre las redes neuronales? ¿Qué **limitaciones prácticas** tiene este teorema?

Respuesta:

El **Teorema de Aproximación Universal** establece que una red neuronal feed-forward con una sola capa oculta, que use una función de activación no lineal adecuada (por ejemplo, sigmoide o ReLU), puede aproximar **cualquier función continua** definida en un conjunto compacto con un error arbitrariamente pequeño. Es decir, existe un número suficientemente grande de neuronas en la capa oculta tal que la red puede representar funciones de alta complejidad.

Sin embargo, este teorema presenta varias **limitaciones prácticas**:

- El teorema es **existencial**: asegura que *existe* una red capaz de aproximar la función, pero no indica cómo encontrar sus parámetros, ni garantiza que el entrenamiento converja.
- La cantidad de neuronas requerida para lograr una buena aproximación puede ser **muy grande**, lo que hace el modelo difícil de entrenar y propenso al sobreajuste.
- No considera aspectos de **generalización**: aunque la red pueda aproximar funciones complejas, no garantiza buen desempeño en datos no vistos.
- No aborda la **optimización**: funciones de pérdida no convexas, mínimos locales y problemas de gradiente pueden impedir encontrar una buena solución.

En resumen, el teorema garantiza capacidad expresiva, pero no asegura eficiencia, entrenabilidad ni generalización en la práctica.

(f) ¿Qué es un **prior conjugado** y por qué es **útil** en el enfoque **bayesiano**?

Respuesta:

Un **prior conjugado** es una distribución previa que, al combinarse con la verosimilitud mediante la Regla de Bayes, produce una distribución posterior perteneciente a la **misma familia funcional** que la del prior. Es decir, si la verosimilitud $p(Y | \theta)$ pertenece a una cierta familia y se elige un prior $p(\theta)$ conjugado, entonces la posterior $p(\theta | Y)$ tendrá la misma forma funcional que $p(\theta)$.

Este concepto es **útil** en el enfoque bayesiano por varias razones:

- **Simplifica los cálculos analíticos:** permite obtener distribuciones posteriores en forma cerrada, evitando métodos numéricos o aproximaciones complejas.
- **Eficiencia computacional:** facilita actualizaciones secuenciales del conocimiento cuando llegan nuevos datos, pues la forma funcional del prior se mantiene.
- **Interpretabilidad:** los parámetros del prior y del posterior se actualizan de manera intuitiva (por ejemplo, conteos en el caso de distribuciones Beta-Binomial).

Un ejemplo clásico es la combinación de una verosimilitud binomial con un prior Beta, donde la posterior también resulta ser una distribución Beta.

(g) ¿Qué **ventajas/desventajas** tiene la **Regresión No Lineal** sobre la **Regresión Lineal**? Nombre 1 de cada una. ¿En qué casos recomienda usted utilizar una Regresión Lineal por sobre una Regresión No Lineal? ¿Y viceversa?

Respuesta:

La **Regresión No Lineal** permite modelar relaciones más complejas entre las variables, mientras que la **Regresión Lineal** asume una relación lineal entre predictores y respuesta.

Ventajas y desventajas:

- **Ventaja de la Regresión No Lineal:** puede capturar patrones y dependencias complejas que una relación lineal no puede representar.
- **Desventaja de la Regresión No Lineal:** mayor riesgo de *sobreajuste* y mayor complejidad computacional.

¿Cuándo usar Regresión Lineal?

- Cuando se espera una relación aproximadamente lineal entre las variables.
- Cuando se necesitan modelos simples, interpretables y robustos.
- Cuando la cantidad de datos es limitada y se quiere evitar sobreajuste.

¿Cuándo usar Regresión No Lineal?

- Cuando la relación entre predictores y respuesta es compleja o claramente no lineal.
- Cuando se dispone de suficientes datos para ajustar un modelo más flexible sin sobreajustar.
- Cuando mejorar la precisión es más importante que la interpretabilidad.

- (h) ¿Qué componente/s debiese tener un **modelo de clasificación**, en su entrenamiento, para poder abordar el problema de **desbalanceo de clases**?

Respuesta:

Para abordar el **desbalanceo de clases** durante el entrenamiento de un modelo de clasificación, es necesario incorporar componentes que compensen la presencia de clases minoritarias. Entre los enfoques más comunes se encuentran:

- **Ponderación de clases (class weighting):** modificar la función de pérdida asignando un mayor peso a los errores cometidos en la clase minoritaria. Esto obliga al modelo a prestar mayor atención a estas observaciones.
- **Re-muestreo del conjunto de entrenamiento:**
 - **Oversampling** de la clase minoritaria (por ejemplo, SMOTE).
 - **Undersampling** de la clase mayoritaria.
- **Funciones de pérdida especializadas:** usar pérdidas como *focal loss*, que penalizan con mayor fuerza los ejemplos difíciles o minoritarios.
- **Umbral de decisión ajustados:** modificar el *threshold* posterior al entrenamiento para equilibrar precisión y recall en clases desbalanceadas.

Cualquiera de estos componentes, o una combinación de ellos, permite entrenar modelos más sensibles a las clases minoritarias y mejorar el desempeño en contextos de desbalance severo.

- (i) Discuta la **importancia de la regularización en modelos de clasificación lineales**. Explique las diferencias entre las **técnicas de regularización L1 y L2**.

Respuesta:

La **regularización** es fundamental en modelos de clasificación lineales para evitar el **sobreajuste**, especialmente cuando el número de características es grande o existe colinealidad entre ellas. Al agregar un término de penalización a la función de pérdida, se controla la magnitud de los coeficientes del modelo, promoviendo soluciones más estables y generalizables.

Las principales técnicas de regularización son:

- **Regularización L2 (Ridge):**

$$\text{Pérdida} = L(\theta) + \lambda \sum_{j=1}^p \theta_j^2,$$

donde $L(\theta)$ es la función de pérdida original y λ controla la fuerza de la regularización. L2 tiende a reducir los coeficientes hacia cero de manera gradual, pero raramente los hace exactamente cero, manteniendo todas las características en el modelo.

- **Regularización L1 (Lasso):**

$$\text{Pérdida} = L(\theta) + \lambda \sum_{j=1}^p |\theta_j|,$$

que puede llevar algunos coeficientes a ser exactamente cero, produciendo un efecto de **selección de variables** implícita y favoreciendo modelos más parcimoniosos.

En resumen, la regularización mejora la generalización del modelo y previene sobreajuste; L2 es útil cuando se desea mantener todas las variables pero reducir su influencia, mientras que L1 es preferible cuando se busca un modelo más interpretable y con selección de características.

- (j) ¿Por qué es **importante el preprocesamiento de datos**, como la **normalización** o **estandarización** de variables, en los métodos de clasificación? Proporcione ejemplos de **algoritmos que son sensibles a la escala** de las variables.

Respuesta:

El **preprocesamiento de datos**, en particular la **normalización** o **estandarización** de variables, es importante porque muchos métodos de clasificación dependen de medidas de distancia o de la magnitud de los coeficientes. Si las variables tienen escalas muy diferentes, aquellas con mayor rango dominarán el cálculo de distancias o gradientes, sesgando el entrenamiento y afectando la convergencia y el desempeño del modelo.

Ejemplos de algoritmos sensibles a la escala de las variables:

- **K-Nearest Neighbors (KNN):** utiliza distancias entre puntos; variables con mayor magnitud pesan más.
- **Support Vector Machines (SVM):** la magnitud de los coeficientes del hiperplano depende de la escala de las características.
- **Redes neuronales:** la escala afecta la actualización de pesos durante el entrenamiento y la convergencia del gradiente.
- **Clustering basado en distancias**, como **k-means**, donde la asignación de clústers depende de la distancia euclidiana.

Por lo tanto, estandarizar (media 0, desviación estándar 1) o normalizar (llevar a un rango $[0,1]$) las variables asegura que todas contribuyan de manera equilibrada al aprendizaje.

- (k) ¿Qué problema se aborda con la **validación cruzada**?

Respuesta:

La **validación cruzada** se utiliza para abordar el problema de **sobreajuste** y para obtener una estimación más confiable del desempeño de un modelo en datos no vistos. Al dividir el conjunto de datos en múltiples subconjuntos (*folds*) y entrenar y evaluar el modelo de manera rotativa, se consigue:

- Evaluar la capacidad de generalización del modelo, reduciendo el sesgo de evaluación que podría ocurrir al usar un solo conjunto de validación.
- Ajustar hiperparámetros de manera más robusta, ya que la performance se mide sobre diferentes particiones de los datos.
- Maximizar el uso de los datos disponibles, especialmente útil cuando el conjunto de entrenamiento es pequeño.

En resumen, la validación cruzada permite estimar de manera más estable y precisa cómo se comportará el modelo ante datos nuevos, mitigando el riesgo de sobreajuste.

- (l) Explique cómo se relaciona el **dilema sesgo-varianza** con el **sobreajuste** y el **subajuste**. ¿Cómo influye en la elección de modelos de **alta complejidad** frente a **modelos más simples**?

Respuesta:

El **dilema sesgo-varianza** describe el trade-off entre dos fuentes de error en un modelo de aprendizaje:

- **Sesgo:** error sistemático debido a suposiciones demasiado fuertes sobre la relación entre variables. Modelos con alto sesgo tienden a **subajustar**, ya que no capturan la complejidad del patrón subyacente.
- **Varianza:** error debido a la sensibilidad del modelo a pequeñas fluctuaciones en los datos de entrenamiento. Modelos con alta varianza tienden a **sobreajustar**, aprendiendo ruido en lugar de la señal general.

La relación con el **sobreajuste** y **subajuste** es directa:

Subajuste $\rightarrow$ alto sesgo, baja varianza, Sobreajuste $\rightarrow$ bajo sesgo, alta varianza.

En la elección de modelos, este dilema influye así:

- **Modelos de alta complejidad** (muchos parámetros, redes profundas, árboles sin poda) tienen baja capacidad de sesgo pero alta varianza. Pueden ajustarse muy bien a los datos de entrenamiento, pero sufren del riesgo de sobreajustarse.
- **Modelos más simples** (regresión lineal, árboles poco profundos) tienen mayor sesgo y menor varianza. Son más robustos pero pueden subajustar si la relación entre variables es compleja.

Por ello, seleccionar el modelo adecuado implica balancear sesgo y varianza para minimizar el error total de generalización.

- (m) La formulación original de **SVM (SVM lineal)** no es aplicable en dos casos: Cuando las clases se **traslapan o se acercan** y cuando la **frontera de decisión no es lineal**. ¿Cómo se puede **extender SVM** para abordar exitosamente estos casos?

Respuesta:

Para extender el **SVM lineal** y manejar los casos donde las clases se traslapan o la frontera no es lineal, se utilizan las siguientes estrategias:

- **Clases traslapadas (soft margin SVM):** Se permite que algunos puntos violen el margen mediante variables de holgura $\xi_i \geq 0$, modificando el problema de optimización a:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i, \quad \text{s.a. } y_i(\mathbf{w}^\top x_i + b) \geq 1 - \xi_i.$$

Aquí, C controla el compromiso entre maximizar el margen y minimizar los errores de clasificación, permitiendo que el modelo sea robusto frente a ruido y traslapes.

- **Frontera no lineal (kernel trick):** Se proyectan los datos a un espacio de mayor dimensión mediante una función de mapeo $\phi(x)$ donde la separación sea lineal:

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle.$$

Esto permite aplicar SVM lineal en el espacio transformado, logrando fronteras de decisión no lineales en el espacio original. Ejemplos comunes de kernels son el **RBF (Gaussian)**, polinomial y sigmoide.

Combinando **soft margin** y **kernel trick**, SVM puede manejar tanto traslapes como relaciones no lineales entre clases de manera efectiva.

- (n) ¿Cuál es el objetivo principal del Análisis de Componentes Principales (ACP) en la reducción de dimensionalidad? Explique brevemente cómo se logra este objetivo y qué limitaciones tiene.

Respuesta:

El objetivo principal del **Análisis de Componentes Principales (ACP)** es reducir la dimensionalidad de un conjunto de datos preservando la mayor cantidad posible de **varianza** presente en las variables originales. Esto facilita la visualización, el almacenamiento y el procesamiento, al tiempo que elimina redundancias y ruido.

Cómo se logra:

- Se calcula la **matriz de covarianza** de los datos estandarizados.
- Se obtienen los **autovalores y autovectores** de esta matriz.
- Se ordenan los autovectores según los autovalores descendentes y se seleccionan los primeros k componentes principales que capturan la mayor parte de la varianza.
- Los datos se proyectan sobre estos k componentes, obteniendo una representación de menor dimensión.

Limitaciones:

- ACP es un método **lineal** y no captura relaciones no lineales entre variables.
- La interpretabilidad de los componentes puede ser difícil, ya que son combinaciones lineales de todas las variables originales.
- Requiere que los datos estén **escalados adecuadamente**, de lo contrario, las variables con mayor magnitud dominarán la varianza.

P2. Verdadero o Falso

Determine si las siguientes afirmaciones son verdaderas o falsas y justifique **brevemente**. (0.6 ptos c/u)

- (a) En entrenamiento, el **error cuadrático medio** de una **regresión lineal regularizada** es mayor al error cuadrático medio de una regresión lineal sin regularización.

Respuesta: Verdadero.

La regularización agrega un término de penalización a la función de pérdida que restringe los coeficientes del modelo. Esto generalmente incrementa el **error de entrenamiento** (como el error cuadrático medio), porque el modelo no puede ajustarse tan perfectamente a los datos de entrenamiento como una regresión lineal sin regularización. Sin embargo, la regularización reduce la **varianza** y mejora la **generalización** a datos nuevos.

- (b) El **modelo de mezcla de gaussianas** puede ser considerada una **generalización de K-means**.

Respuesta: Verdadero.

El modelo de mezcla de gaussianas (GMM) puede considerarse una generalización de K-means porque K-means corresponde al caso límite de GMM cuando todas las componentes gaussianas tienen varianza igual y covarianza diagonal idéntica, y la asignación de puntos a clústers es dura (cada punto pertenece a un solo clúster). GMM permite asignaciones **probabilísticas** (suaves) y clústers elípticos con diferentes tamaños y formas, aumentando la flexibilidad del modelo.

- (c) El **estimador de mínimos cuadrados ordinarios** coincide con el **estimador máximo verosímil** cuando se considera un **modelo lineal con ruido gaussiano**.

Respuesta: Verdadero.

Cuando el modelo lineal se asume con ruido gaussiano $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$, la función de verosimilitud del modelo es proporcional a

$$L(\theta) \propto \exp\left(-\frac{1}{2\sigma^2}\|Y - X\theta\|^2\right).$$

Maximizar la verosimilitud equivale a minimizar la suma de los cuadrados de los errores $\|Y - X\theta\|^2$, que es exactamente el criterio de los **mínimos cuadrados ordinarios (OLS)**. Por lo tanto, OLS coincide con el estimador máximo verosímil bajo ruido gaussiano.

- (d) Un ensamble tipo **Bagging** reduce la **varianza de los modelos de base**.

Respuesta: Verdadero.

El **Bagging** (Bootstrap Aggregating) construye múltiples modelos de base sobre diferentes subconjuntos de los datos generados mediante muestreo con reemplazo y luego promedia sus predicciones (o vota en clasificación). Esto reduce la **varianza** del modelo final, ya que los errores individuales de cada modelo tienden a compensarse, manteniendo el sesgo aproximadamente igual pero mejorando la estabilidad y la generalización.

- (e) El problema de **clustering** es una instancia de **aprendizaje supervisado**.

Respuesta: Falso.

El **clustering** es un problema de **aprendizaje no supervisado**, ya que no se dispone de etiquetas de clase para guiar el entrenamiento. El objetivo es agrupar los datos en clústers según su similitud intrínseca, sin información previa sobre categorías o salidas deseadas.

- (f) Un **modelo de clustering** puede usarse **exitosamente** para resolver el problema de **clasificación**.

Respuesta: Falso en general.

Un **modelo de clustering** agrupa datos sin usar etiquetas, mientras que la **clasificación** requiere aprender una función que asigne correctamente cada dato a una clase conocida. Aunque en algunos casos los clústers coincidan aproximadamente con las clases, esto no garantiza resultados precisos ni generalizables, por lo que clustering no es una solución confiable para problemas de clasificación supervisada.

- (g) **K-means** es un algoritmo de **clustering jerárquico** que no requiere especificar el **número de clusters de antemano**.

Respuesta: Falso.

K-means no es un algoritmo de clustering jerárquico; es un método de **partición** que divide los datos en un número **fijo de clústers** K , el cual debe especificarse de antemano. Los algoritmos jerárquicos, como *agglomerative clustering*, construyen un árbol de clústers y pueden permitir decidir el número de clústers a posteriori.

- (h) El **Análisis de Componentes Principales (PCA)** es una técnica de **aprendizaje supervisado** utilizada para **reducir la dimensionalidad** de los datos.

Respuesta: Falso.

El **PCA** es una técnica de **aprendizaje no supervisado**, ya que no utiliza etiquetas de clase para su cálculo. Su objetivo es reducir la dimensionalidad de los datos proyectándolos sobre componentes principales que capturan la mayor parte de la varianza, pero no busca optimizar la predicción de una variable de salida.

- (i) Las **redes neuronales recurrentes** son **útiles** para problemas de **predicción secuenciales**.

Respuesta: Verdadero.

Las **redes neuronales recurrentes (RNN)** están diseñadas para manejar datos secuenciales, ya que mantienen un estado interno que captura información de pasos anteriores. Esto las hace especialmente útiles para problemas como series temporales, procesamiento de lenguaje natural o predicción de secuencias, donde el contexto histórico influye en la predicción futura.

- (j) Sea X una variable aleatoria en $\mathbb{R}^D$ e Y variable aleatoria a valores $\mathbb{R}$. Considere el modelo de regresión lineal $Y = \beta^T X + \varepsilon$, con $\varepsilon \sim N(0, \sigma_\varepsilon^2)$, con $\beta = (\beta_1, \dots, \beta_D)^T$. Asuma σ_ε^2 conocido y que X es independiente de β y de ε . Considere el **estimador máximo a posteriori** usando el **prior gaussiano** $p(\beta) = N(0; \sigma_\beta^2 I)$. A medida que σ_β^2 tiende a cero el modelo correspondiente al máximo a posteriori tendrá una **regularización más débil**.

Respuesta: Falso.

En el estimador máximo a posteriori (MAP) con un **prior gaussiano** $\beta \sim N(0, \sigma_\beta^2 I)$, el término de regularización es equivalente a una penalización L2:

$$\text{Pérdida MAP} = \|Y - X\beta\|^2 + \frac{\sigma_\varepsilon^2}{\sigma_\beta^2} \|\beta\|^2.$$

A medida que $\sigma_\beta^2 \rightarrow 0$, el coeficiente $\frac{\sigma_\varepsilon^2}{\sigma_\beta^2}$ crece, aumentando la fuerza de la regularización. Por lo tanto, **una varianza menor del prior produce regularización más fuerte, no más débil**.

P3. Preguntas teóricas

- (a) Considere una **distribución geométrica** con probabilidad de éxito p_0 , dada por:

$$P(X = k | p_0) = (1 - p_0)^{k-1} p_0, \quad k = 1, 2, 3, \dots$$

donde $p_0 \in (0, 1)$. Considere una **distribución Beta** con parámetros $\alpha > 0$ y $\beta > 0$, cuya densidad está definida como:

$$f(p_0 | \alpha, \beta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} p_0^{\alpha-1} (1 - p_0)^{\beta-1}, \quad 0 < p_0 < 1.$$

donde la **función Gamma** se define como:

$$\Gamma(z) = \int_0^\infty t^{z-1} e^{-t} dt, \quad z > 0.$$

Una propiedad de la función Γ es la siguiente:

$$\Gamma(n) = (n - 1)!, \quad n \in \mathbb{N}.$$

- Derive la **distribución posterior** $f(p_0 | \{X_1, \dots, X_n\})$ después de observar una muestra $X_1, X_2, \dots, X_n$ de la distribución geométrica. Muestre que la distribución posterior sigue siendo una distribución Beta y determine sus **nuevos parámetros**.

Respuesta:

Sea $X_1, \dots, X_n$ una muestra independiente de la distribución geométrica con parámetro p_0 . La **verosimilitud** es:

$$L(p_0 | X_1, \dots, X_n) = \prod_{i=1}^n P(X_i | p_0) = \prod_{i=1}^n (1 - p_0)^{X_i-1} p_0 = p_0^n (1 - p_0)^{\sum_{i=1}^n (X_i-1)}.$$

El **prior** Beta es:

$$f(p_0 | \alpha, \beta) \propto p_0^{\alpha-1} (1 - p_0)^{\beta-1}.$$

La **distribución posterior** se obtiene aplicando la regla de Bayes:

$$f(p_0 | X_1, \dots, X_n) \propto L(p_0 | X_1, \dots, X_n) f(p_0 | \alpha, \beta).$$

Sustituyendo:

$$f(p_0 | X_1, \dots, X_n) \propto p_0^n (1 - p_0)^{\sum_{i=1}^n (X_i-1)} p_0^{\alpha-1} (1 - p_0)^{\beta-1} = p_0^{\alpha+n-1} (1 - p_0)^{\beta+\sum_{i=1}^n (X_i-1)-1}.$$

Esto corresponde a la **forma de una distribución Beta**:

$$\text{Posterior: } p_0 | X_1, \dots, X_n \sim \text{Beta}(\alpha', \beta'),$$

donde los **nuevos parámetros** son:

$$\alpha' = \alpha + n, \quad \beta' = \beta + \sum_{i=1}^n (X_i - 1) = \beta + \sum_{i=1}^n X_i - n.$$

Por lo tanto, la distribución posterior sigue siendo Beta, con parámetros actualizados por la información de la muestra.

- (b) Supongamos que tenemos un modelo de **regresión lineal regularizado** en una dimensión, donde x es la variable independiente e y es la variable dependiente:

$$\arg \min_{\theta} \|y - x^T \theta\|_2^2 + \rho \|\theta\|_p^p.$$

- (a) Para el caso **unidimensional** y con $p = 2$, el **parámetro óptimo de mínimos cuadrados** está dado por

$$\hat{\theta} = \arg \min_{\theta} (y \cdot x - \theta)^2 + \rho \cdot \theta^2. \quad (1)$$

Demuestre, que:

$$\hat{\theta} = (x^2 + \rho)^{-1} x \cdot y$$

cumple la condición de minimalidad de primer orden, i.e., soluciona la ecuación (1).

Respuesta:

La función a minimizar es:

$$f(\theta) = (y \cdot x - \theta)^2 + \rho \theta^2.$$

Derivamos con respecto a θ y igualamos a cero para la condición de minimalidad de primer orden:

$$\frac{df}{d\theta} = 2(\theta - x \cdot y) + 2\rho\theta = 0.$$

Resolviendo para θ :

$$2(\theta - x \cdot y) + 2\rho\theta = 0 \implies \theta(1 + \rho) = x \cdot y \implies \hat{\theta} = \frac{x \cdot y}{x^2 + \rho}.$$

Por lo tanto, $\hat{\theta} = (x^2 + \rho)^{-1} x \cdot y$ cumple la condición de primer orden y minimiza la función.

- (b) En el **caso general** del modelo de regresión lineal, es decir, **múltiples puntos de entrenamiento** y **multidimensional**, señale qué sucede con la **varianza del estimador** en los siguientes casos:

- Fijar $p \geq 1$ y **aumentar** ρ .
- Fijar ρ y considerar los casos $p = 1$ y $p = 2$.

Respuesta:

- **Fijar $p \geq 1$ y aumentar ρ :** Aumentar ρ incrementa la fuerza de regularización, lo que penaliza coeficientes grandes y reduce la **varianza del estimador**. Sin embargo, esto puede incrementar el sesgo del modelo (trade-off sesgo-varianza).
- **Fijar ρ y comparar $p = 1$ y $p = 2$:**
 - $p = 2$ (Ridge): la regularización L2 reduce la varianza de manera uniforme, pero rara vez produce coeficientes exactamente cero.

- $p = 1$ (Lasso): la regularización L1 puede establecer algunos coeficientes exactamente en cero, reduciendo la complejidad del modelo y la varianza de los coeficientes no nulos, pero de manera menos suave que L2.

En resumen, aumentar ρ o usar regularización L1/L2 disminuye la varianza del estimador, mientras que la elección de p determina el patrón de reducción y la sparsity de los coeficientes.

(c) Considere la **función kernel polinomial**:

$$K : \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}, \quad K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + \gamma)^p, \quad \text{con } p = 2, \gamma = 1.$$

Además, sea:

$$\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^6, \quad \mathbf{x} \mapsto (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2).$$

Demuestre que:

$$K(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle.$$

Respuesta:

Sea $\mathbf{x} = (x_1, x_2)^T$ y $\mathbf{y} = (y_1, y_2)^T$. La **función kernel polinomial** con $p = 2$ y $\gamma = 1$ es:

$$K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + 1)^2 = (x_1y_1 + x_2y_2 + 1)^2.$$

Expandimos el cuadrado:

$$(x_1y_1 + x_2y_2 + 1)^2 = 1 + 2x_1y_1 + 2x_2y_2 + x_1^2y_1^2 + x_2^2y_2^2 + 2x_1x_2y_1y_2.$$

Por otro lado, el mapeo $\phi(\mathbf{x})$ está dado por:

$$\phi(\mathbf{x}) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2),$$

y su producto interno con $\phi(\mathbf{y})$ es:

$$\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle = 1 \cdot 1 + (\sqrt{2}x_1)(\sqrt{2}y_1) + (\sqrt{2}x_2)(\sqrt{2}y_2) + x_1^2y_1^2 + x_2^2y_2^2 + (\sqrt{2}x_1x_2)(\sqrt{2}y_1y_2).$$

Simplificando los factores $\sqrt{2}$:

$$\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle = 1 + 2x_1y_1 + 2x_2y_2 + x_1^2y_1^2 + x_2^2y_2^2 + 2x_1x_2y_1y_2.$$

Por lo tanto:

$$K(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle.$$

Esto demuestra que el kernel polinomial de grado 2 con $\gamma = 1$ corresponde al producto interno en el espacio transformado definido por ϕ .

- (d) Considere una **red neuronal feed-forward** $\mathcal{N}_2 : \mathbb{R}^2 \rightarrow \mathbb{R}$ con una **sola capa oculta** con **dos neuronas** con una **función de activación ReLU** y **capa de salida lineal**. Pruebe que existen pesos tales que $\mathcal{N}_2(x_1, x_2) = -x_2$, para todo $(x_1, x_2) \in \mathbb{R}^2$.

Respuesta:

Consideremos una red neuronal feed-forward $\mathcal{N}_2 : \mathbb{R}^2 \rightarrow \mathbb{R}$ con una capa oculta de dos neuronas con activación ReLU $\sigma(z) = \max(0, z)$ y una capa de salida lineal. Sea la representación de la red:

$$\mathcal{N}_2(x_1, x_2) = w_1 \sigma(u_{11}x_1 + u_{12}x_2 + b_1) + w_2 \sigma(u_{21}x_1 + u_{22}x_2 + b_2) + b_{\text{out}},$$

donde u_{ij} son los pesos de la capa oculta, b_i los sesgos, w_i los pesos de salida y b_{out} el sesgo de salida. Para representar la función lineal $\mathcal{N}_2(x_1, x_2) = -x_2$, podemos elegir pesos que hagan que la ReLU se comporte linealmente sobre todo $\mathbb{R}^2$. Por ejemplo:

$$\left\{ \begin{array}{l} u_{11} = 0 \\ u_{12} = -1 \\ b_1 = 0 \\ w_1 = 1 \\ u_{21} = 0 \\ u_{22} = 1 \\ b_2 = 0 \\ w_2 = -1 \\ b_{\text{out}} = 0 \end{array} \right.$$

Entonces, la salida de la red es:

$$\begin{aligned} \mathcal{N}_2(x_1, x_2) &= w_1 \sigma(-x_2) + w_2 \sigma(x_2) \\ &= 1 \cdot \max(0, -x_2) + (-1) \cdot \max(0, x_2) \\ &= -x_2, \quad \forall x_2 \in \mathbb{R}. \end{aligned}$$

De esta manera, la red puede representar exactamente la función deseada usando dos neuronas ReLU en la capa oculta y una salida lineal.

P4. Preguntas de aplicación

- (a) Asuma que entrena un **modelo de clasificación SVM** usando un **kernel polinomial** $K(\mathbf{x}_i, \mathbf{x}_j) = (\langle \mathbf{x}_i, \mathbf{x}_j \rangle + c)^p$, para varios valores de $p \in \{1, 2, 3, 4\}$. Asuma que el parámetro c se escoge usando **validación-cruzada**. Se muestran tres posibles resultados de su error (**mean square error**) tanto de **entrenamiento** como **testeo**:

1.

p	1	2	3	4
Train error	0.36	0.31	0.24	0.17
Test error	0.25	0.21	0.18	0.23

2.

p	1	2	3	4
Train error	0.16	0.23	0.28	0.33
Test error	0.29	0.24	0.31	0.35

3.

p	1	2	3	4
Train error	0.24	0.20	0.18	0.13
Test error	0.32	0.26	0.23	0.28

(a)

¿Cuál resultado es **más realista** de encontrar en la práctica? Justifique.

Respuesta:

El resultado **más realista** es el 1.

- En la práctica, a medida que aumentamos el grado del kernel polinomial p , el modelo se vuelve más complejo y flexible. Esto típicamente reduce el **error de entrenamiento** de manera continua, como se observa en la fila "Train error" del resultado 1: $0.36 \rightarrow 0.17$. - El **error de test** inicialmente disminuye al aumentar p (de 0.25 a 0.18), indicando que un modelo más complejo se ajusta mejor a los datos y generaliza mejor. - Sin embargo, al aumentar demasiado la complejidad ($p = 4$), el error de test aumenta a 0.23, lo que indica **sobreajuste**: el modelo se ajusta demasiado al ruido del entrenamiento y su generalización empeora.

Los resultados 2 y 3 son menos realistas:

- En el resultado 2, el error de entrenamiento aumenta con la complejidad, lo cual contradice la teoría de que modelos más flexibles ajustan mejor los datos de entrenamiento. - En el resultado 3, el error de test disminuye hasta cierto punto, pero luego aumenta de manera irregular, y el sobreajuste se ve demasiado marcado sin una tendencia suave, lo cual es menos típico en práctica.

Por lo tanto, el patrón de disminución del error de entrenamiento combinado con un mínimo del error de test seguido de un ligero aumento (como en el resultado 1) refleja un comportamiento típico de **trade-off sesgo-varianza** en problemas reales.

- (b) Observe la siguiente imagen. ¿Cuál es la **variable dependiente** y cuál es la **independiente**? ¿Tiene sentido **intercambiar** las variables dependiente e independiente? ¿Qué **modelo** se está utilizando en este caso?

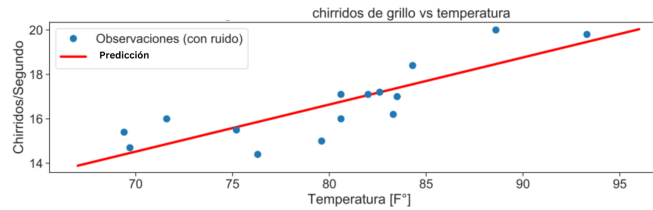


Figura 1: Enter Caption

Respuesta:

- **Variable independiente (X):** Temperatura ($T[°F]$).
- **Variable dependiente (Y):** Chirridos/Segundo del grillo.
- **Intercambio de variables:** No tiene sentido, ya que la relación causal biológica es Temperatura $\rightarrow$ Chirridos; asumir lo contrario carece de fundamento.
- **Modelo:** Regresión Lineal Simple, con ecuación

$$Y = \beta_0 + \beta_1 X + \varepsilon,$$

donde Y son los chirridos y X la temperatura.

Ahora observe el siguiente gráfico, donde se presenta la predicción anterior como Predicción 1 y una nueva Predicción 2 considerando un nuevo punto (denotado $\star$).

- Comente por qué afecta la introducción del punto $\star$ a la predicción y cómo podría la esta ser robusta a la introducción del outlier. Es decir, cómo, a pesar de $\star$ podría Predicción 2 ser similar a Predicción 1.

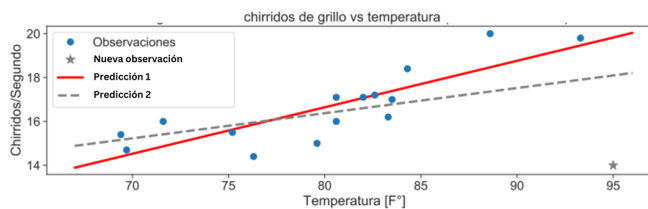


Figura 2: Enter Caption

Respuesta:

El punto $\star$ es un **outlier** que afecta la Predicción 2 porque la regresión lineal por **mínimos cuadrados ordinarios (OLS)** penaliza fuertemente errores grandes, desplazando la línea de ajuste.

Para que la Predicción 2 sea similar a la Predicción 1, se pueden aplicar estrategias de **robustez**:

- **Regresión robusta:** Usar estimadores M (por ejemplo, función de Huber) o regresión L1 (mínimos errores absolutos), que penalizan menos los outliers.
- **Regularización:** Emplear Ridge o Lasso con un parámetro de penalización alto para limitar cambios en los coeficientes.
- **Detección/eliminación de outliers:** Identificar puntos con alto residuo o leverage y excluirllos antes de entrenar.

Considere ahora la siguiente imagen que muestra 3 predicciones correspondientes a un mismo modelo en el que se ha variado un parámetro de su entrenamiento. Responda **brevemente** las preguntas.

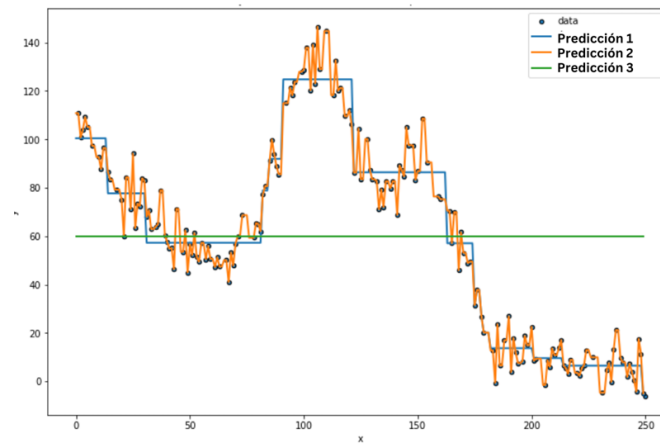


Figura 3

- (c) ¿Qué **modelo** se está utilizando? Explique qué **parámetro del modelo varía** entre las tres estimaciones.

Respuesta:

- El modelo utilizado es probablemente un **Árbol de Decisión para Regresión** (o un modelo basado en árboles como un *Random Forest* o *Gradient Boosting*). Se identifica por la forma **escalonada** y **discontinua** de las predicciones 1 y 2.
- El parámetro que varía entre las tres estimaciones es la **complejidad del modelo** o **profundidad máxima del árbol** (`max_depth`) o el **número mínimo de muestras por hoja** (`min_samples_leaf`). La **Predicción 1** tiene una complejidad **alta** (muchas divisiones/poca regularización); la **Predicción 3** tiene una complejidad **mínima** (pocas divisiones/mucha regularización).

- (d) ¿Cual de las tres predicciones le parece la **más apropiada**?

Respuesta:

- La **Predicción 2** es la más apropiada.
- La **Predicción 1** presenta **sobreajuste** (*overfitting*) ya que sigue demasiado de cerca el ruido de los datos de entrenamiento.

- La **Predicción 3** presenta **subajuste** (*underfitting*) ya que es demasiado simple y no logra capturar la tendencia principal de los datos.
- La **Predicción 2** logra un **equilibrio sesgo-varianza** razonable, capturando la tendencia general sin memorizar el ruido.

(e) ¿A qué corresponde el valor de la estimación para la **predicción 3**?

Respuesta:

- La Predicción 3 es una **línea horizontal constante**. Corresponde al valor **promedio** ($\bar{y}$) de todos los puntos de datos en el conjunto de entrenamiento.
- Esto ocurre cuando el árbol de decisión tiene la **complejidad mínima** posible (e.g., $\text{max_depth} = 1$ o $\text{min_samples_leaf} = n$, donde n es el número total de datos), resultando en una única hoja que predice el promedio global.

(f) ¿Cual de los tres elegiría para realizar una **poda de costo-complejidad**?

Respuesta:

- Elegiría la **Predicción 1**.
- La **Poda de Costo-Complejidad** (*Cost-Complexity Pruning* o *Weakest Link Pruning*) se aplica sobre un árbol que ha sido entrenado hasta tener un **sobreajuste máximo** (como la Predicción 1), y luego se lo simplifica sistemáticamente para encontrar el mejor **sub-árbol** con base en la validación cruzada.

(g) ¿Cual es el **error en el conjunto de entrenamiento** para la **predicción 2**?

Respuesta:

- El error de entrenamiento para la Predicción 2 (*Train Error*) es **mayor** que el error de entrenamiento para la Predicción 1, pero **menor** que el error de entrenamiento para la Predicción 3.
- La Predicción 1 (sobreajuste) tiene el **mínimo error de entrenamiento** porque se ajusta estrechamente a cada punto.
- La Predicción 3 (subajuste) tiene el **máximo error de entrenamiento** porque es muy simple.
- La Predicción 2, al ser un modelo con complejidad intermedia, tendrá un error de entrenamiento que cumple con:

$$Error_{Train}(\text{Predicción 1}) < Error_{Train}(\text{Predicción 2}) < Error_{Train}(\text{Predicción 3})$$